
Clever Compiler Error Reference

Small Basic Plus — for young programmers

 A comprehensive error reference for the Clever compiler (Small Basic Plus) designed for young programmers. 161 errors with translations, explanations, and code examples. Perfect for lessons, robotics clubs, and self-learning.

version 1.0

status in progress

errors 161

How to Use This Guide

1. **Read the error message** in the Clever output window.
 2. **Find the error number** in this guide.
 3. **Read the explanation** — it tells you in simple words what went wrong.
 4. **Look at the example** — it shows the INCORRECT code and the CORRECT code.
 5. **Fix your program** by following the example.
-

For Compiler Errors (#1–#5):

1. Open the folder containing your program (`.bp` file).
 2. Find the **temporary folder** named `~your_program_name` (e.g., `~robot`).
 3. Open the file `~your_program_name.bp` inside that folder in Notepad or any text editor.
 4. Find the line and position shown by the compiler (e.g., `123:45`).
 5. Look at the code there — that's where the error is!
 6. Go back to your original `.bp` file and fix the error in the same place.
-

Table of Contents

Block 0: Compiler Errors (#1–#5)

- [#1: Unexpected Symbol](#)
- [#2: Unexpected ID](#)
- [#3: Unexpected EOL](#)
- [#4: Expected Expression](#)
- [#5: Expected Statement](#)

Block 1: Syntax Errors (#6–#25)

- [#6: Extra bracket](#)
- [#7: Brackets not closed properly](#)
- [#8: Invalid number of parameters](#)
- [#9: Invalid number of parameters or math operator missing](#)
- [#10: Invalid math operator](#)
- [#11: Different data types](#)
- [#12: Excess math operators](#)
- [#13: Missing parameter](#)
- [#14: Method does not return values](#)
- [#15: Method does not return values as a parameter](#)
- [#16: Invalid parameter type](#)
- [#17: Incorrect method definition](#)
- [#18: Method not found](#)
- [#19: Extra brackets in method call](#)
- [#20: Missing brackets in method call](#)
- [#21: File name must be a string](#)
- [#22: Parameters must be indicated as a string](#)
- [#23: Row not recognized](#)
- [#24: Row type not recognized](#)
- [#25: Errors in the row / Invalid expressions in the row](#)

Block 2: Structure Errors (IF, FOR, WHILE, SUB, FUNCTION) (#26–#43)

- #26: IF structure is not closed / not closed properly
- #27: IF structure has no beginning
- #28: FOR structure is not closed / not closed properly
- #29: FOR structure has no beginning
- #30: WHILE structure is not closed / not closed properly
- #31: WHILE structure has no beginning
- #32: SUB structure is not closed
- #33: SUB structure has no beginning
- #34: FUNCTION structure is not closed
- #35: FUNCTION has no beginning
- #36: SUB cannot contain another SUB
- #37: SUB cannot contain FUNCTION
- #38: FUNCTION cannot contain SUB
- #39: FUNCTION cannot contain another FUNCTION
- #40: Invalid code — only EndIf / EndFor / EndWhile / EndSub / EndFunction / Else
- #41: Invalid code — only EndFunction
- #42: Missing keyword 'To'
- #43: Errors in For initialization line

Block 3: Variable and Type Errors (#44–#57)

- #44: No value assigned to variable
- #45: Variable not defined
- #46: Variable not initialized
- #47: Variable has different type
- #48: Incorrect variable definition
- #49: Missing variable name
- #50: Character '=' missing after variable name definition
- #51: Character '=' must be followed by an expression
- #52: Invalid math operators in variable definition
- #53: Invalid expressions in variable definition
- #54: '++' can only be applied to numeric variables
- #55: '--' can only be applied to numeric variables
- #56: Only a number can be written to variable with minus sign

- #57: Strings can only be added with '+' operator

● **Block 4: Label and Goto Errors (#58–#62)**

- #58: Invalid code — only Goto and label name
- #59: Invalid code — only label name and colon
- #60: Label with this name is already defined in this function
- #61: Label with this name is already defined in this program
- #62: Label not found in function / program

● **Block 5: Procedure and Function Errors (#63–#89)**

- #63: Procedure not found
- #64: Function not found
- #65: Invalid procedure definition
- #66: Procedure definition must contain 'Sub' and a name
- #67: Missing procedure name in procedure definition
- #68: Procedure definition contains invalid keywords
- #69: Procedure definition contains invalid expressions
- #70: Procedure with this name is already defined
- #71: Invalid procedure call from module syntax
- #72: Missing brackets in procedure call
- #73: Error in procedure call parameters
- #74: Several commas in a row in procedure call parameters
- #75: Invalid expressions in procedure call parameters
- #76: Invalid math operators in procedure call parameters
- #77: Missing parameter after comma in procedure call
- #78: Invalid function definition
- #79: Missing brackets in function definition
- #80: Missing function name in function definition
- #81: Invalid keywords in function definition
- #82: Invalid expressions in function definition
- #83: Function definition contains variables with the same name
- #84: Name cannot be used — function with this name is already defined
- #85: Name cannot be used — procedure with this name is already defined

- #86: Function with this name and number of parameters is already defined
- #87: Variable definitions must be separated with a comma in function definition
- #88: There must be a variable before the comma in function definition
- #89: Variable type not defined in function definition

● **Block 6: Array Errors (#90–#99)**

- #90: Mathematical operations cannot be performed on arrays
- #91: Only integers can be used for array indexing
- #92: Only numbers can be used for array indexing
- #93: This operation cannot be applied to arrays
- #94: Array index must be an integer (without fractional part)
- #95: Only integer variables can be used for array indexing
- #96: A method that returns a number can be used for array indexing
- #97: Another array cannot be written to an array element
- #98: Invalid values in array index
- #99: Only methods that return numbers or strings can be used with +=, -=, *=, /=

● **Block 7: Math and Logical Operator Errors (#100–#107)**

- #100: An operand is expected after math operator
- #101: An operand is expected before math operator
- #102: An operand is expected between math operators
- #103: Math operator is expected before the next operand
- #104: Boolean expression must have two operands
- #105: There must be a logical operator between operands
- #106: Only numbers and strings can be compared
- #107: You cannot use two logical operators in a row

○ **Block 8: Include, folder, global Errors (#108–#115)**

- #108: Module not found
- #109: File not found
- #110: Missing name of file being included
- #111: Included files cannot contain their own inclusions
- #112: Included files cannot contain the keyword 'folder'

- #113: Keyword 'folder' can only be declared once
- #114: Keyword 'folder' cannot be used in module files
- #115: Keyword 'global' cannot be used in module files

◆ Block 9: Project Errors (#116–#121)

- #116: Project name cannot contain more than 32 characters
- #117: Project name cannot be void
- #118: Project name must begin with A-Z, a-z
- #119: Project name can only contain letters, numbers, and _
- #120: The first parameter must be "prjs" or "sd"
- #121: Keyword 'folder' must be before the main code

◆ Block 10: .bpm Module and Extended Construct Errors (#122–#171)

- #122: Missing variable type after 'in/out'
- #123: Variable type must be specified before the variable
- #124: 'in/out' must follow a comma
- #125: 'in/out' must be before the variable type
- #126: Variable name must follow the variable type
- #127: 'in/out' and type must precede the variable
- #128: Variable type not defined in function definition
- #129: Parameter has different type
- #130: Output parameter in function call must be a variable
- #131: Global variables cannot be used as parameters
- #132: Boolean expression missing left side
- #133: Boolean expression missing right side
- #134: Errors in For initialization line
- #135: 'For' must be followed by a variable and assignment
- #136: In For loop, variable must contain a number
- #137: 'Then' must be at the end of the line
- #138: Missing logical condition
- #139: Missing imported module file name
- #140: Imported modules cannot contain 'include'
- #141: Imported modules cannot contain keyword 'folder'

- #142: Procedures are not allowed in .bpm files
- #143: .bpm files can only contain functions and properties
- #144: Global variables cannot be used in .bpm
- #145: Global labels cannot be used in .bpm
- #146: Module property must consist of type and name
- #147: Module property: type and name
- #148: Property type can only be number, number[], string, string[]
- #149: Property with this name is already defined
- #150: Property cannot be declared inside a module method
- #151: Only 'private' keyword is allowed on this line
- #152: Private property can only be called in the owning module
- #153: Private method can only be called in the owning module
- #154: Variable name conflicts with property name in module
- #155: Only one 'Break' per line
- #156: 'Break' can only be used inside For and While loops
- #157: Only one 'Continue' per line
- #158: 'Continue' can only be used inside For and While loops
- #159: Only one 'Return' per line
- #160: 'Return' can only be used inside Sub and Function
- #161: Property with this name is not defined in the module

BLOCK 0: COMPILER ERRORS (#1–#5)

These errors occur **after preprocessing**. The line numbers shown **do not match** your `.bp` file!

How to fix any error in this block:

1. Open the folder with your program.
2. Find the folder `~program_name`.
3. Open the file `~program_name.bp` inside it in Notepad.
4. Find the line and position (e.g., `123:45`).
5. Look at the code — that's where the error is!

6. Go back to your `.bp` file and fix it there.

Error #1: Unexpected Symbol

Compiler says:

```
Unexpected Symbol at 123:45
```

What it means in simple words:

The compiler found a character that shouldn't be there (e.g., `@`, `#`, `$`, or an extra comma).

How to fix it:

- 1. Check for extra characters in the line.
- 2. Make sure commas and periods are placed correctly.
- 3. Don't use Cyrillic letters in variable names.

Incorrect:

```
x = 5 @ 3
```

Correct:

```
x = 5 + 3
```

Error #2: Unexpected ID

Compiler says:

```
Unexpected ID at 123:45
```

What it means in simple words:

The compiler found an identifier (variable, label, or command name) where it didn't expect one. Often this means you forgot an operator or a keyword like `Then` or `To`.

How to fix it:

1. Check if you forgot **Then** in an **If** statement.
2. Check if you forgot **To** in a **For** loop.
3. Check if a variable name is next to a number without an operator.

Incorrect:

```
x = 5 3
```

Correct:

```
x = 5 + 3
```

Error #3: Unexpected EOL

Compiler says:

```
Unexpected EOL at 123:45
```

What it means in simple words:

The compiler expected more code (like a closing quote or parenthesis) but found the end of the line instead.

How to fix it:

1. Make sure all strings are closed with quotes.
2. Make sure parentheses are balanced.
3. Make sure you didn't end a line with an operator.

Incorrect:

```
LCD.Write(10, 10, "Hello
```

Correct:

```
LCD.Write(10, 10, "Hello")
```

Error #4: Expected Expression

Compiler says:

Expected Expression at 123:45

What it means in simple words:

The compiler expected an expression (number, variable, or method call) but found nothing.

How to fix it:

1. Make sure there's something after `=`.
2. Make sure operators have operands on both sides.

Incorrect:

```
x = 5 +
```

Correct:

```
x = 5 + 3
```

Error #5: Expected Statement

Compiler says:

Expected Statement at 123:45

What it means in simple words:

The compiler expected a command (statement) but found something else. Often an empty line or an unfinished structure.

How to fix it:

1. Make sure there's a command after `Then`.
2. Check for empty lines where code should be.

Incorrect:

```
If x > 5 Then
```

Correct:

```
If x > 5 Then  
    LCD.Write(10, 10, "Bigger")  
EndIf
```

BLOCK 1: SYNTAX ERRORS (#6–#25)

Error #6: Extra bracket

Compiler says:

Extra bracket

What it means in simple words:

You added a bracket where it's not needed.

How to fix it:

1. Count your opening and closing brackets — they must match.

Incorrect:

```
x = (5 + 3))
```

Correct:

```
x = (5 + 3)
```

Error #7: Brackets not closed properly

Compiler says:

Brackets not closed properly

What it means in simple words:

You opened a bracket but forgot to close it.

How to fix it:

1. Every opening bracket needs a closing one.

Incorrect:

```
x = (5 + 3
```

Correct:

```
x = (5 + 3)
```

Error #8: Invalid number of parameters

Compiler says:

Invalid number of parameters

What it means in simple words:

You passed the wrong number of arguments to a method or procedure.

How to fix it:

1. Check how many parameters the method expects.
2. Compare with what you are passing.

Incorrect:

```
LCD.Clear("a")
```

Correct:

```
LCD.Clear()
```

Error #9: Invalid number of parameters or math operator missing

Compiler says:

Invalid number of **parameters** or math operator missing

What it means in simple words:

Something is wrong with parameters or operators in the line.

How to fix it:

1. Check the number of parameters.
2. Make sure operators are present.

Incorrect:

```
x = 5 3
```

Correct:

```
x = 5 + 3
```

Error #10: Invalid math operator

Compiler says:

Invalid math **operator**

What it means in simple words:

You used an unsupported mathematical symbol.

How to fix it:

1. Use only allowed operators: +, -, *, /.

Incorrect:

```
x = 5 ^ 3
```

Correct:

```
x = Math.Power(5, 3)
```

Error #11: Different data types

Compiler says:

Different data types

What it means in simple words:

You're mixing types that cannot be combined (e.g., adding a number to a string).

How to fix it:

1. Use only numbers with numbers and strings with strings.

Incorrect:

```
x = 5 + "Hello"
```

Correct:

```
x = 5 + 3  
x = "Hello" + " world"
```

Error #12: Excess math operators

Compiler says:

Excess math operators

What it means in simple words:

You wrote multiple operators in a row.

How to fix it:

1. Use only one operator between values.

Incorrect:

```
x = 5 ++ 3
```

Correct:

```
x = 5 + 3
```

Error #13: Missing parameter

Compiler says:

```
Missing parameter
```

What it means in simple words:

You called a method but didn't pass the required parameter.

How to fix it:

1. Add the missing parameter.

Incorrect:

```
a = Math.Abs()
```

Correct:

```
a = Math.Abs(-5)
```

Error #14: Method does not return values

Compiler says:

```
Method does not return values
```

What it means in simple words:

You tried to use a method as a value, but it doesn't return anything.

How to fix it:

1. Use only functions (**FUNCTION**) in expressions.
2. Do not use procedures (**SUB**) as values.

Incorrect:

```
x = LCD.Write(10, 10, "Hello")
```

Correct:

```
LCD.Write(10, 10, "Hello")  
x = 5
```

Error #15: Method does not return values as a parameter

Compiler says:

```
Method does not return values as a parameter
```

What it means in simple words:

You used a method as a parameter for another method, but it doesn't return anything.

How to fix it:

1. Only use methods that return a value.

Incorrect:

```
x = Math.Sin(LCD.Write(10, 10, "Hello"))
```

Correct:

```
x = Math.Sin(5)  
LCD.Write(10, 10, "Hello")
```

Error #16: Invalid parameter type

Compiler says:

```
Invalid parameter type
```

What it means in simple words:

You passed a parameter of the wrong type.

How to fix it:

1. Check the expected type for the method.
2. Pass the correct type.

Incorrect:

```
Math.Sin("5")
```

Correct:

```
Math.Sin(5)
```

Error #17: Incorrect method definition

Compiler says:

```
Incorrect method definition
```

What it means in simple words:

You declared a method (function) incorrectly.

How to fix it:

1. Check the syntax: `Function` + name + `()`.

Incorrect:

```
MyFunction
```

Correct:

```
Function MyFunction()  
    Return 5  
EndFunction
```

Error #18: Method not found

Compiler says:

```
Method not found
```

What it means in simple words:

You're trying to call a method that Clever doesn't recognize.

How to fix it:

1. Check the spelling.
2. Check the number of parameters.

Incorrect:

```
LCD.Write(10, "Hello")
```

Correct:

```
LCD.Write(10, 10, "Hello")
```

Error #19: Extra brackets in method call

Compiler says:

```
Extra brackets in method call
```

What it means in simple words:

You added extra parentheses around the method arguments.

How to fix it:

1. Remove the extra brackets.

Incorrect:

```
LCD.Write((10, 10, "Hello"))
```

Correct:

```
LCD.Write(10, 10, "Hello")
```

Error #20: Missing brackets in method call

Compiler says:

```
Missing brackets in method call
```

What it means in simple words:

You called a method without parentheses.

How to fix it:

1. Add `()` after the method name.

Incorrect:

```
LCD.Clear
```

Correct:

```
LCD.Clear()
```

Error #21: File name must be a string

Compiler says:

```
File name must be a string
```

What it means in simple words:

You passed a file name as a number or something else instead of text in quotes.

How to fix it:

1. Put the file name in quotes.

Incorrect:

```
Include data
```

Correct:

```
Include "data"
```

Error #22: Parameters must be indicated as a string

Compiler says:

```
Parameters must be indicated as a string
```

What it means in simple words:

You passed a parameter as a number, but it should be text in quotes.

How to fix it:

1. Put the parameter in quotes.

Incorrect:

```
folder prjs MyProject
```

Correct:

```
folder "prjs" "MyProject"
```

Error #23: Row not recognized

Compiler says:

```
Row not recognized
```

What it means in simple words:

The compiler didn't understand what you wrote in that line.

How to fix it:

1. Check the spelling of the command.
2. Make sure the line isn't empty.

Incorrect:

```
Print "Hello"
```

Correct:

```
LCD.Write(10, 10, "Hello")
```

Error #24: Row type not recognized

Compiler says:

```
Row type not recognized
```

What it means in simple words:

The compiler can't tell if this line is a command, variable, label, or something else.

How to fix it:

1. Check the syntax.
2. Avoid using keywords as variable names.

Incorrect:

```
If x > 5  
    LCD.Write(10, 10, "Bigger")
```

Correct:

```
If x > 5 Then  
    LCD.Write(10, 10, "Bigger")  
EndIf
```

Error #25: Errors in the row / Invalid expressions in the row

Compiler says:

Errors in the row
Invalid expressions in the row

What it means in simple words:

There are one or more errors in this line.

How to fix it:

1. Check the whole line for typos.
2. Make sure quotes, brackets, and punctuation are correct.

Incorrect:

```
x = 5 + * 3  
x = 5 @ 3
```

Correct:

```
x = 5 * 3  
x = 5 + 3
```

BLOCK 2: STRUCTURE ERRORS (IF, FOR, WHILE, SUB, FUNCTION) (#26–#43)

Error #26: IF structure is not closed / not closed properly

Compiler says:

```
IF structure is not closed
IF structure not closed properly
```

What it means in simple words:

You opened an `IF` block but forgot to close it with `EndIf`.

How to fix it:

1. Make sure every `IF` has its own `EndIf`.

Incorrect:

```
If x > 5 Then
    LCD.Write(10, 10, "Bigger")
```

Correct:

```
If x > 5 Then
    LCD.Write(10, 10, "Bigger")
EndIf
```

Error #27: IF structure has no beginning

Compiler says:

```
IF structure has no beginning
```

What it means in simple words:

You wrote `EndIf` but there is no matching `IF` before it.

How to fix it:

1. Make sure every `EndIf` has a matching `IF`.

Incorrect:

```
EndIf  
x = 5
```

Correct:

```
If x > 5 Then  
    x = 5  
EndIf
```

Error #28: FOR structure is not closed / not closed properly

Compiler says:

```
FOR structure is not closed  
FOR structure not closed properly
```

What it means in simple words:

You opened a **FOR** loop but forgot to close it with **EndFor**.

How to fix it:

1. Make sure every **FOR** has its own **EndFor**.

Incorrect:

```
For i = 1 To 10  
    LCD.Write(10, 10, i)
```

Correct:

```
For i = 1 To 10  
    LCD.Write(10, 10, i)  
EndFor
```

Error #29: FOR structure has no beginning

Compiler says:

```
FOR structure has no beginning
```

What it means in simple words:

You wrote `EndFor` but there is no matching `FOR` before it.

How to fix it:

1. Make sure every `EndFor` has a matching `FOR` .

Incorrect:

```
EndFor  
LCD.Write(10, 10, "Ready")
```

Correct:

```
For i = 1 To 10  
    LCD.Write(10, 10, "Ready")  
EndFor
```

Error #30: WHILE structure is not closed / not closed properly

Compiler says:

```
WHILE structure is not closed  
WHILE structure not closed properly
```

What it means in simple words:

You opened a `WHILE` loop but forgot to close it with `EndWhile` .

How to fix it:

1. Make sure every `WHILE` has its own `EndWhile` .

Incorrect:

```
While x < 10  
    x = x + 1
```

Correct:

```
While x < 10  
    x = x + 1  
EndWhile
```

Error #31: WHILE structure has no beginning

Compiler says:

WHILE structure has no beginning

What it means in simple words:

You wrote `EndWhile` but there is no matching `WHILE` before it.

How to fix it:

1. Make sure every `EndWhile` has a matching `WHILE`.

Incorrect:

```
EndWhile  
x = x + 1
```

Correct:

```
While x < 10  
    x = x + 1  
EndWhile
```

Error #32: SUB structure is not closed

Compiler says:

SUB structure is not closed

What it means in simple words:

You started a procedure with `Sub` but forgot to close it with `EndSub`.

How to fix it:

1. Make sure every `Sub` has its own `EndSub`.

Incorrect:

```
Sub MyProcedure
  LCD.Write(10, 10, "Bigger")
```

Correct:

```
Sub MyProcedure
  LCD.Write(10, 10, "Bigger")
EndSub
```

Error #33: SUB structure has no beginning

Compiler says:

SUB structure has no beginning

What it means in simple words:

You wrote `EndSub` but there is no matching `Sub` before it.

How to fix it:

1. Make sure every `EndSub` has a matching `Sub`.

Incorrect:

```
LCD.Write(10, 10, "Hello")
EndSub
```

Correct:

```
Sub MyProcedure
    LCD.Write(10, 10, "Hello")
EndSub
```

Error #34: FUNCTION structure is not closed

Compiler says:

```
FUNCTION structure is not closed
```

What it means in simple words:

You started a function but forgot to close it with `EndFunction` .

How to fix it:

1. Make sure every function has its own `EndFunction` .

Incorrect:

```
Function MyFunction()
    a = 5
```

Correct:

```
Function MyFunction()
    a = 5
EndFunction
```

Error #35: FUNCTION has no beginning

Compiler says:

```
FUNCTION has no beginning
```

What it means in simple words:

You wrote `EndFunction` but there is no matching `Function` before it.

How to fix it:

1. Make sure every `EndFunction` has a matching `Function`.

Incorrect:

```
a = 5
EndFunction
```

Correct:

```
Function MyFunction()
    a = 5
EndFunction
```

Error #36: SUB cannot contain another SUB

Compiler says:

```
SUB structure can not contain other SUB structure
```

What it means in simple words:

You tried to declare a procedure (`SUB`) inside another procedure.

How to fix it:

1. Move the inner procedure outside.

Incorrect:

```
Sub Main
    LCD.Write(10, 10, "First")
    Sub Inner
        LCD.Write(10, 10, "Second")
    EndSub
EndSub
```

Correct:

```
Sub Main
    LCD.Write(10, 10, "First")
EndSub

Sub Inner
    LCD.Write(10, 10, "Second")
EndSub
```

Error #37: SUB cannot contain FUNCTION

Compiler says:

SUB structure can not contain other FUNCTION structure

What it means in simple words:

You tried to declare a function (**Function**) inside a procedure (**Sub**).

How to fix it:

1. Move the function outside.

Incorrect:

```
Sub MyProcedure
    LCD.Write(10, 10, "First")
    Function MyFunction()
        a = 5
    EndFunction
EndSub
```

Correct:

```
Sub MyProcedure
    LCD.Write(10, 10, "First")
EndSub
```

```
Function MyFunction()  
    a = 5  
EndFunction
```

Error #38: FUNCTION cannot contain SUB

Compiler says:

FUNCTION structure can not contain other SUB structure

What it means in simple words:

You tried to declare a procedure (**Sub**) inside a function (**Function**).

How to fix it:

1. Move the procedure outside.

Incorrect:

```
Function MyFunction()  
    a = 5  
    Sub MyProcedure  
        LCD.Write(10, 10, "Hello")  
    EndSub  
EndFunction
```

Correct:

```
Function MyFunction()  
    a = 5  
EndFunction  
  
Sub MyProcedure  
    LCD.Write(10, 10, "Hello")  
EndSub
```

Error #39: FUNCTION cannot contain another FUNCTION

Compiler says:

FUNCTION structure can not contain other FUNCTION structure

What it means in simple words:

You tried to declare a function inside another function.

How to fix it:

1. Move the inner function outside.

Incorrect:

```
Function Main()  
    a = 5  
    Function Inner()  
        b = 10  
    EndFunction  
EndFunction
```

Correct:

```
Function Main()  
    a = 5  
EndFunction  
  
Function Inner()  
    b = 10  
EndFunction
```

Error #40: Invalid code — only EndIf / EndFor / EndWhile / EndSub / EndFunction / Else

Compiler says:

Invalid code. 'EndIf' must **only be** indicated
Invalid code. 'EndFor' must **only be** indicated
Invalid code. 'EndWhile' must **only be** indicated
Invalid code. 'EndSub' must **only be** indicated
Invalid code. 'Else' must **only be** indicated

What it means in simple words:

You wrote extra text on the same line as one of these keywords.

How to fix it:

1. The line must contain only the keyword.

Incorrect:

```
EndIf x = 5
```

Correct:

```
EndIf
```

Error #41: Invalid code — only EndFunction

Compiler says:

Invalid code. 'EndFunction' must **only be** indicated

What it means in simple words:

You wrote extra text on the same line as `EndFunction`.

How to fix it:

1. The line must contain only `EndFunction`.

Incorrect:

```
EndFunction MyFunction()
```

Correct:

EndFunction

Error #42: Missing keyword 'To'

Compiler says:

Missing keyword 'To'

What it means in simple words:

You forgot the **To** keyword between the start and end values in a **For** loop.

How to fix it:

1. Always write **To** in the **For** line.

Incorrect:

```
For i = 1 10
    LCD.Write(10, 10, i)
EndFor
```

Correct:

```
For i = 1 To 10
    LCD.Write(10, 10, i)
EndFor
```

Error #43: Errors in For initialization line

Compiler says:

Errors in For initialization line

What it means in simple words:

There is a syntax error in the **For** start line.

How to fix it:

1. Use the correct format: `For variable = start_value To end_value`.

Incorrect:

```
For i 1 To 10  
For i = 1 10
```

Correct:

```
For i = 1 To 10  
    LCD.Write(10, 10, i)  
EndFor
```

BLOCK 3: VARIABLE AND TYPE ERRORS (#44–#57)

Error #44: No value assigned to variable

Compiler says:

No value assigned to `variable`

What it means in simple words:

You declared a variable but forgot to assign a value to it.

How to fix it:

1. Use `=` to assign a value.

Incorrect:

```
x
```

Correct:

```
x = 5
```

Error #45: Variable not defined

Compiler says:

```
Variable not defined
```

What it means in simple words:

You are using a variable that hasn't been declared yet.

How to fix it:

1. Declare the variable before using it.

Incorrect:

```
y = x + 5
```

Correct:

```
x = 10  
y = x + 5
```

Error #46: Variable not initialized

Compiler says:

```
Variable not initialized
```

What it means in simple words:

You declared a variable but didn't give it a value before using it.

How to fix it:

1. Assign a value before using the variable.

Incorrect:

```
number x
y = x + 5
```

Correct:

```
x = 10
y = x + 5
```

Error #47: Variable has different type

Compiler says:

```
Variable has different type
```

What it means in simple words:

You are performing an operation with a variable containing the wrong data type.

How to fix it:

1. Check what the variable contains.
2. Convert the type if needed.

Incorrect:

```
x = "Hello"
y = x + 5
```

Correct:

```
x = 10
y = x + 5
```

Error #48: Incorrect variable definition

Compiler says:

Incorrect `variable` definition

What it means in simple words:

You wrote a variable declaration incorrectly.

How to fix it:

1. Use the format: `name = value`.

Incorrect:

```
x 5
```

Correct:

```
x = 5
```

Error #49: Missing variable name

Compiler says:

Missing `variable name`

What it means in simple words:

You wrote `=` but didn't put a variable name on the left.

How to fix it:

1. Add a variable name before `=`.

Incorrect:

```
= 5
```

Correct:

```
x = 5
```

Error #50: Character '=' missing after variable name definition

Compiler says:

Character '=' missing after variable name definition

What it means in simple words:

You wrote the variable name but forgot = .

How to fix it:

1. Add = after the variable name.

Incorrect:

x 5

Correct:

x = 5

Error #51: Character '=' must be followed by an expression

Compiler says:

Character '=' must be followed by an expression

What it means in simple words:

You wrote = but there is nothing after it.

How to fix it:

1. Write a number, variable, or expression after = .

Incorrect:

x =

Correct:

```
x = 5
```

Error #52: Invalid math operators in variable definition

Compiler says:

```
Invalid math operators in variable definition
```

What it means in simple words:

You used an incorrect operator when defining a variable.

How to fix it:

1. Use only one operator between numbers.

Incorrect:

```
x = 5 ++ 3
```

Correct:

```
x = 5 + 3
```

Error #53: Invalid expressions in variable definition

Compiler says:

```
Invalid expressions in variable definition
```

What it means in simple words:

The expression on the right side of `=` doesn't make sense to the compiler.

How to fix it:

1. Check the expression syntax.

Incorrect:


```
x = (5 +
```

Correct:

```
x = 5 + 3
```

Error #54: '++' can only be applied to numeric variables

Compiler says:

```
Mathematical operation '++' can only be applied to numeric variables
```

What it means in simple words:

You tried to increment a variable that contains text.

How to fix it:

1. Make sure the variable contains a number.

Incorrect:

```
x = "Hello"  
x++
```

Correct:

```
x = 5  
x++
```

Error #55: '--' can only be applied to numeric variables

Compiler says:

```
Mathematical operation '--' can only be applied to numeric variables
```

What it means in simple words:

You tried to decrement a variable that contains text.

How to fix it:

1. Make sure the variable contains a number.

Incorrect:

```
x = "Hello"  
x--
```

Correct:

```
x = 5  
x--
```

Error #56: Only a number can be written to variable with minus sign**Compiler says:**

```
Only a number can be written to variable with minus sign
```

What it means in simple words:

You tried to assign a negative value that is not a number.

How to fix it:

1. Use the minus sign only with numbers.

Incorrect:

```
x = -"Hello"
```

Correct:

```
x = -5
```

Error #57: Strings can only be added with '+' operator

Compiler says:

Rows can only be added with '+' operator

What it means in simple words:

You tried to use an unsupported operator on strings.

How to fix it:

1. Use `+` to concatenate strings.

Incorrect:

```
x = "Hello" - "world"
```

Correct:

```
x = "Hello" + " world"
```

BLOCK 4: LABEL AND GOTO ERRORS (#58–#62)

Error #58: Invalid code — only Goto and label name

Compiler says:

Invalid code. 'Goto' and jump label name must only be indicated

What it means in simple words:

You have extra text on the line with `Goto`.

How to fix it:

1. The line must contain only `Goto` + label name.

Incorrect:

```
Goto :myLabel  
Goto myLabel:
```

Correct:

```
Goto myLabel
```

Error #59: Invalid code — only label name and colon

Compiler says:

Invalid code. Jump label name and two-spot at the end must only be indicated

What it means in simple words:

You have extra text on the line where you define a label.

How to fix it:

1. The label definition must contain only the name and a colon.

Incorrect:

```
myLabel: LCD.Write(10, 10, "Hello")
```

Correct:

```
myLabel:  
LCD.Write(10, 10, "Hello")
```

Error #60: Label with this name is already defined in this function

Compiler says:

Label with this name is already defined in this function

What it means in simple words:

You're trying to create two labels with the same name inside one function.

How to fix it:

1. Use a different name for the label.

Incorrect:

```
Sub Test
  start:
  LCD.Write(10, 10, "1")
  start:
  LCD.Write(20, 10, "2")
EndSub
```

Correct:

```
Sub Test
  start:
  LCD.Write(10, 10, "1")
  nextStep:
  LCD.Write(20, 10, "2")
EndSub
```

Error #61: Label with this name is already defined in this program

Compiler says:

```
Label with this name is already defined in this program
```

What it means in simple words:

You're trying to create two labels with the same name in the same program.

How to fix it:

1. Use a different name for the label.

Incorrect:

```
start:
Goto next
start:
```

Correct:

```
start:
Goto next
nextStep:
```

Error #62: Label not found in function / program

Compiler says:

```
No label with this name found in the function
No label with this name found in the program
```

What it means in simple words:

You wrote `Goto` but the label doesn't exist.

How to fix it:

1. Check the spelling of the label.
2. Make sure the label is declared.

Incorrect:

```
Goto missingLabel
```

Correct:

```
myLabel:
Goto myLabel
```

Error #63: Procedure not found

Compiler says:

```
Procedure not found
```

What it means in simple words:

You're calling a procedure that doesn't exist in the program.

How to fix it:

1. Check the spelling.
2. Check the number of parameters.

Incorrect:

```
Function MyFunction(in number a)
    LCD.Write(10, 10, a)
EndFunction

MyFunction()
```

Correct:

```
Function MyFunction(in number a)
    LCD.Write(10, 10, a)
EndFunction

MyFunction(1)
```

Error #64: Function not found

Compiler says:

```
Function not found
```

What it means in simple words:

You're calling a function that doesn't exist.

How to fix it:

1. Check the spelling.
2. Check the number of parameters.

Incorrect:

```
Function MyFunction(in number a)
    LCD.Write(10, 10, a)
EndFunction

MyFunction()
```

Correct:

```
Function MyFunction(in number a)
    LCD.Write(10, 10, a)
EndFunction

MyFunction(1)
```

Error #65: Invalid procedure definition

Compiler says:

```
Invalid procedure definition
```

What it means in simple words:

You wrote a procedure incorrectly.

How to fix it:

1. Use the correct syntax: `Sub` + name.

Incorrect:

Sub

Correct:

```
Sub MyProcedure
    LCD.Write(10, 10, "Hello")
EndSub
```

Error #66: Procedure definition must contain 'Sub' and a name

Compiler says:

Procedure definition must contain keyword 'Sub' and procedure name

What it means in simple words:

You forgot **Sub** or the procedure name.

How to fix it:

1. Start the procedure with **Sub** and a valid name (not a keyword).

Incorrect:

```
Sub include
```

Correct:

```
Sub MyProcedure
```

Error #67: Missing procedure name in procedure definition

Compiler says:

Missing procedure name in procedure definition

What it means in simple words:

You wrote `Sub` but didn't give the procedure a name.

How to fix it:

1. Add a name after `Sub`.

Incorrect:

```
Sub
```

Correct:

```
Sub MyProcedure
```

Error #68: Procedure definition contains invalid keywords

Compiler says:

```
Procedure definition contains invalid keywords
```

What it means in simple words:

You used the wrong keyword for declaring a procedure.

How to fix it:

1. Use only `Sub` for procedures.

Incorrect:

```
Sub Function
```

Correct:

```
Sub MyProcedure
```

Error #69: Procedure definition contains invalid expressions

Compiler says:

```
Procedure definition contains invalid expressions
```

What it means in simple words:

There is extra text on the procedure declaration line.

How to fix it:

1. Leave only `Sub` and the procedure name.

Incorrect:

```
Sub MyProcedure = 5
```

Correct:

```
Sub MyProcedure
```

Error #70: Procedure with this name is already defined

Compiler says:

```
Procedure with this name is already defined
```

What it means in simple words:

You're trying to create a procedure with a name that's already used.

How to fix it:

1. Use a different name.

Incorrect:

```
Sub MyProcedure
    ...
EndSub
Sub MyProcedure
```

```
...  
EndSub
```

Correct:

```
Sub MyProcedure  
...  
EndSub  
Sub AnotherProcedure  
...  
EndSub
```

Error #71: Invalid procedure call from module syntax

Compiler says:

```
Invalid procedure call from module syntax
```

What it means in simple words:

You're calling a procedure from a module incorrectly.

How to fix it:

1. Use the syntax: `ModuleName.ProcedureName()` .

Incorrect:

```
MyProcedure()
```

Correct:

```
MyModule.MyProcedure()
```

Error #72: Missing brackets in procedure call

Compiler says:

Missing brackets `in procedure call`

What it means in simple words:

You called a procedure without parentheses.

How to fix it:

1. Add `()` after the procedure name.

Incorrect:

```
MyProcedure
```

Correct:

```
MyProcedure()
```

Error #73: Error in procedure call parameters

Compiler says:

Error `in procedure call parameters`

What it means in simple words:

Something is wrong with the parameters you passed.

How to fix it:

1. Check the number and types of parameters.

Incorrect:

```
MyProcedure(5, "Hello")
```

Correct:

```
MyProcedure(5)
```

Error #74: Several commas in a row in procedure call parameters

Compiler says:

Several commas **in** a row **in procedure call parameters**

What it means in simple words:

You placed two commas in a row in the parameter list.

How to fix it:

1. Remove the extra comma.

Incorrect:

```
MyProcedure(5,, 3)
```

Correct:

```
MyProcedure(5, 3)
```

Error #75: Invalid expressions in procedure call parameters

Compiler says:

Invalid expressions **in procedure call parameters**

What it means in simple words:

You passed an incorrectly written expression as a parameter.

How to fix it:

1. Check the expression syntax.

Incorrect:

```
MyProcedure(5 +)
```

Correct:

```
MyProcedure(5 + 3)
```

Error #76: Invalid math operators in procedure call parameters

Compiler says:

```
Invalid math operators in procedure call parameters
```

What it means in simple words:

You used unsupported operators in the parameters.

How to fix it:

1. Use only allowed operators.

Incorrect:

```
MyProcedure(5 ++ 3)
```

Correct:

```
MyProcedure(5 + 3)
```

Error #77: Missing parameter after comma in procedure call

Compiler says:

```
Missing parameter after the decimal point in procedure call
```

What it means in simple words:

You placed a comma but didn't list the next parameter.

How to fix it:

1. Remove the extra comma or add the missing parameter.

Incorrect:

```
MyProcedure(5,)
```

Correct:

```
MyProcedure(5, 3)
```

Error #78: Invalid function definition

Compiler says:

```
Invalid function definition
```

What it means in simple words:

You declared a function incorrectly.

How to fix it:

1. Use the correct syntax: `Function` + name + `(parameters)` .

Incorrect:

```
Function
```

Correct:

```
Function MyFunction()  
    a = 5  
EndFunction
```

Error #79: Missing brackets in function definition

Compiler says:

```
Missing brackets in function definition
```


What it means in simple words:

You forgot the parentheses after the function name.

How to fix it:

1. Add `()` after the function name.

Incorrect:

```
Function MyFunction
    a = 5
EndFunction
```

Correct:

```
Function MyFunction()
    a = 5
EndFunction
```

Error #80: Missing function name in function definition

Compiler says:

Missing `function name` in `function definition`

What it means in simple words:

You wrote `Function` but didn't give it a name.

How to fix it:

1. Add a name after `Function`.

Incorrect:

```
Function
    a = 5
EndFunction
```

Correct:

```
Function MyFunction()  
    a = 5  
EndFunction
```

Error #81: Invalid keywords in function definition

Compiler says:

Invalid keywords `in function definition`

What it means in simple words:

You used the wrong keyword or a reserved word as a name.

How to fix it:

1. Use `Function` and avoid keywords as names.

Incorrect:

```
Function in()  
    a = 5  
EndFunction
```

Correct:

```
Function MyIn()  
    a = 5  
EndFunction
```

Error #82: Invalid expressions in function definition

Compiler says:

Invalid expressions `in function definition`

What it means in simple words:

There is extra text on the function declaration line.

How to fix it:

1. Leave only `Function`, name, and `()`.

Incorrect:

```
Function in()  
    a = 5  
EndFunction
```

Correct:

```
Function MyIn()  
    a = 5  
EndFunction
```

Error #83: Function definition contains variables with the same name

Compiler says:

```
Function definition contains variables with the same name
```

What it means in simple words:

You have two parameters with the same name.

How to fix it:

1. Use different names for each parameter.

Incorrect:

```
Function MyFunction(in number a, in number a)  
    x = a  
EndFunction
```

Correct:

```
Function MyFunction(in number a, in number b)
    x = a + b
EndFunction
```

Error #84: Name cannot be used — function with this name is already defined

Compiler says:

Name can not be used because a function with this name is already defined

What it means in simple words:

You're trying to create a function with a name that already exists.

How to fix it:

1. Use a different name.

Incorrect:

```
Function MyFunction()
    a = 5
EndFunction

Function MyFunction()
    a = 10
EndFunction
```

Correct:

```
Function MyFunction()
    a = 5
EndFunction

Function MyOtherFunction()
    a = 10
EndFunction
```

Error #85: Name cannot be used — procedure with this name is already defined

Compiler says:

Name can not be used because a procedure with this name is already defined

What it means in simple words:

You're trying to create a function with a name already used by a procedure.

How to fix it:

1. Use a different name.

Incorrect:

```
Sub MyProcedure
    LCD.Write(10, 10, "Hello")
EndSub

Function MyProcedure()
    a = 5
EndFunction
```

Correct:

```
Sub MyProcedure
    LCD.Write(10, 10, "Hello")
EndSub

Function MyFunction()
    a = 5
EndFunction
```

Error #86: Function with this name and number of parameters is already defined

Compiler says:

Function with this name and number of parameters is already defined

What it means in simple words:

You're trying to create a function with the same name and same number of parameters as an existing one.

How to fix it:

1. Use a different name or change the number of parameters.

Incorrect:

```
Function MyFunction(a)
```

```
    x = a
```

```
EndFunction
```

```
Function MyFunction(b)
```

```
    x = b
```

```
EndFunction
```

Correct:

```
Function MyFunction(a)
```

```
    x = a
```

```
EndFunction
```

```
Function MyFunction2(b)
```

```
    x = b
```

```
EndFunction
```

Error #87: Variable definitions must be separated with a comma in function definition

Compiler says:

Variable definitions **in function definition** must be separated **with** a comma

What it means in simple words:

You forgot commas between parameters.

How to fix it:

1. Separate parameters with commas.

Incorrect:

```
Function MyFunction(in number a in number b in number c)
    x = a + b + c
EndFunction
```

Correct:

```
Function MyFunction(in number a, in number b, in number c)
    x = a + b + c
EndFunction
```

Error #88: There must be a variable before the comma in function definition

Compiler says:

There must be **a variable before the comma in function definition**

What it means in simple words:

You placed a comma but there is no parameter before it.

How to fix it:

1. Make sure every comma is preceded by a parameter.

Incorrect:

```
Function MyFunction(, in number b, in number c)
    x = b + c
EndFunction
```

Correct:

```
Function MyFunction(in number a, in number b, in number c)
    x = a + b + c
EndFunction
```

Error #89: Variable type not defined in function definition

Compiler says:

Variable **type not** defined **in function definition**

What it means in simple words:

You wrote **in** or **out** but forgot the type.

How to fix it:

1. Always specify the type after **in** or **out**.

Incorrect:

```
Function MyFunction(in a)
    x = a
EndFunction
```

Correct:

```
Function MyFunction(in number a)
    x = a
EndFunction
```

BLOCK 6: ARRAY ERRORS (#90–#99)

Error #90: Mathematical operations cannot be performed on arrays

Compiler says:

`Mathematical` operations can not be performed `on` arrays

What it means in simple words:

You tried to perform an operation on an entire array.

How to fix it:

1. Work with individual array elements.

Incorrect:

```
a[0] = 5  
b[0] = 3  
x = a + b
```

Correct:

```
x = a[0] + b[0]
```

Error #91: Only integers can be used for array indexing

Compiler says:

Only integers can be used for `array` indexing

What it means in simple words:

You used a non-integer value as an index.

How to fix it:

1. Use only integers for indices.

Incorrect:

```
a[5.5] = 10
```

Correct:

```
a[5] = 10
```

Error #92: Only numbers can be used for array indexing

Compiler says:

Only figures can be used for **array** indexing

What it means in simple words:

You used a non-numeric value as an index.

How to fix it:

1. Use only numbers for indices.

Incorrect:

```
a["two"] = 10
```

Correct:

```
a[2] = 10
```

Error #93: This operation cannot be applied to arrays

Compiler says:

This mathematical operations can not **be** applied **to** arrays

What it means in simple words:

You tried to apply an operation to an entire array.

How to fix it:

1. Apply operations to individual elements.

Incorrect:

```
a[0] = 5  
a++
```

Correct:

```
a[0]++
```

Error #94: Array index must be an integer (without fractional part)

Compiler says:

```
Array index number must be an integer (without fractional part)
```

What it means in simple words:

You used a fractional number as an index.

How to fix it:

1. Use only whole numbers as indices.

Incorrect:

```
a[5.7] = 10
```

Correct:

```
a[5] = 10
```

Error #95: Only integer variables can be used for array indexing

Compiler says:

```
Variables containing only integers can be used for array indexing
```

What it means in simple words:

You used a variable as an index, but it doesn't contain an integer.

How to fix it:

1. Make sure the variable contains an integer.

Incorrect:

```
i = 5.7  
a[i] = 10
```

Correct:

```
i = 5  
a[i] = 10
```

Error #96: A method that returns a number can be used for array indexing

Compiler says:

```
Method that returns a number can be used for array indexing
```

What it means in simple words:

You used a method as an index, but it doesn't return a number.

How to fix it:

1. Use only methods that return numbers.

Incorrect:

```
a[LCD.Write(10, 10, "Hello")] = 10
```

Correct:

```
i = Math.Power(2, 3)  
a[i] = 10
```

Error #97: Another array cannot be written to an array element

Compiler says:

Another **array** can **not** be written to an **array** element

What it means in simple words:

You tried to write an entire array into a single element.

How to fix it:

1. Write only a single value to each element.

Incorrect:

```
a[0] = b
```

Correct:

```
a[0] = b[0]
```

Error #98: Invalid values in array index

Compiler says:

Invalid values in **array** index

What it means in simple words:

You used an invalid value as an index (e.g., negative or too large).

How to fix it:

1. Use non-negative integers as indices.

Incorrect:

```
a[-1] = 10
```

Correct:

```
a[0] = 10
```

Error #99: Only methods that return numbers or strings can be used with +=, -=, *=, /=

Compiler says:

Only methods that return figures or rows can be used with operators +=, -=, *=, /=

What it means in simple words:

You used a compound operator with a method that returns nothing.

How to fix it:

1. Use these operators only with methods that return a value.

Incorrect:

```
x += LCD.Clear()
```

Correct:

```
x += 3
```

BLOCK 7: MATH AND LOGICAL OPERATOR ERRORS (#100–#107)

Error #100: An operand is expected after math operator

Compiler says:

An operand (variable, number, method) is expected after math operator

What it means in simple words:

You placed an operator but there's nothing after it.

How to fix it:

1. Add a number, variable, or method after the operator.

Incorrect:

```
x = 5 +
```

Correct:

```
x = 5 + 3
```

Error #101: An operand is expected before math operator

Compiler says:

An operand (variable, number, **method**) is expected before math operator

What it means in simple words:

You placed an operator but there's nothing before it.

How to fix it:

1. Add a number, variable, or method before the operator.

Incorrect:

```
x = + 5
```

Correct:

```
x = 3 + 5
```

Error #102: An operand is expected between math operators

Compiler says:

An operand (variable, number, **method**) **is** expected between math operators

What it means in simple words:

You wrote two operators in a row.

How to fix it:

1. Put a number between the operators.

Incorrect:

```
x = 5 + + 3  
x = 5 * / 3
```

Correct:

```
x = 5 + 3
```

Error #103: Math operator is expected before the next operand

Compiler says:

Math **operator** **is** expected **after** the next operand

What it means in simple words:

You wrote two numbers in a row without an operator.

How to fix it:

1. Put an operator between the numbers.

Incorrect:

```
x = 5 3
```

Correct:

```
x = 5 + 3
```

Error #104: Boolean expression must have two operands

Compiler says:

Boolean expression must have two operands

What it means in simple words:

The logical expression has only one operand.

How to fix it:

1. Add a second operand.

Incorrect:

```
If x > Then
    LCD.Write(10, 10, "Hello")
EndIf
```

Correct:

```
If x > 5 Then
    LCD.Write(10, 10, "x bigger then 5")
EndIf
```

Error #105: There must be a logical operator between operands

Compiler says:

There must be a logical operator between operands

What it means in simple words:

You wrote two operands without a logical operator.

How to fix it:

1. Use a logical operator: `=`, `<>`, `<`, `>`, `<=`, `>=`, `And`, `Or`.

Incorrect:

```
If x 5 Then
    LCD.Write(10, 10, "x = 5")
EndIf
```

Correct:

```
If x = 5 Then
    LCD.Write(10, 10, "x = 5")
EndIf
```

Error #106: Only numbers and strings can be compared

Compiler says:

Numbers **and** lines can only **be** compared

What it means in simple words:

You tried to compare something that isn't a number or string.

How to fix it:

1. Compare only numbers and strings.

Incorrect:

```
If a = b Then
    LCD.Write(10, 10, "the arrays are equal")
EndIf
```

Correct:

```
If a[0] = b[0] Then
    LCD.Write(10, 10, "the elements are equal")
EndIf
```

Error #107: You cannot use two logical operators in a row

Compiler says:

You cannot use two logical operators in a row

What it means in simple words:

You placed two logical operators one after another.

How to fix it:

1. Use only one operator between expressions.

Incorrect:

```
If x > 5 And And y < 10 Then
    LCD.Write(10, 10, "condition exists")
EndIf
```

Correct:

```
If x > 5 And y < 10 Then
    LCD.Write(10, 10, "condition exists")
EndIf
```

BLOCK 8: INCLUDE, FOLDER, GLOBAL ERRORS (#108–#115)

Error #108: Module not found

Compiler says:

Module not found

What it means in simple words:

You're trying to use a module but the compiler can't find it.

How to fix it:

1. Check the file name (do not include extension).

2. Check if the file is in the same folder.
 3. Check the relative path.
 4. Try closing both files and reopening — sometimes the IDE caches old paths.
-

Error #109: File not found

Compiler says:

```
File not found
```

What it means in simple words:

You're trying to open or include a file that doesn't exist.

How to fix it:

1. Check the file name (do not include extension).
 2. Check if the file is in the same folder.
 3. Check the relative path.
 4. Try closing both files and reopening.
-

Error #110: Missing name of file being included

Compiler says:

```
Missing name of file being included
```

What it means in simple words:

You wrote `Include` but didn't specify the file name.

How to fix it:

1. Put the file name in quotes.

Incorrect:

```
Include
```

Correct:

```
Include "myModule"
```

Error #111: Included files cannot contain their own inclusions

Compiler says:

Files being included can **not contain** own inclusions

What it means in simple words:

You're trying to include a file that itself contains an `Include` .

How to fix it:

1. Place all `Include` statements only in the main file.

Incorrect:

```
' main.bp
Include "module1"

' module1
Include "module2"
```

Correct:

```
' main.bp
Include "module1"
Include "module2"
```

Error #112: Included files cannot contain the keyword 'folder'

Compiler says:

Files being included can **not** contain keyword 'folder'

What it means in simple words:

You used `folder` inside an included file.

How to fix it:

1. Move the `folder` declaration to the main file.

Incorrect:

```
' module1
folder "prjs" MyProject"
```

Correct:

```
' main.bp
folder "prjs" "MyProject"
Include "module1"
```

Error #113: Keyword 'folder' can only be declared once

Compiler says:

Keyword 'folder' can **only be** declared once

What it means in simple words:

You're trying to use `folder` more than once.

How to fix it:

1. Keep only one `folder` declaration.

Incorrect:

```
folder "prjs" "Project1"
folder "prjs" "Project2"
```

Correct:

```
folder "prjs" "MyProject"
```

Error #114: Keyword 'folder' cannot be used in module files

Compiler says:

```
Keyword 'folder' can not be used in module files
```

What it means in simple words:

You used `folder` inside a module file.

How to fix it:

1. Move the `folder` declaration to the main file.

Incorrect:

```
' myModule.bpm  
folder "prjs" "MyProject"
```

Correct:

```
' main.bp  
folder "prjs" "MyProject"  
Include "myModule.bpm"
```

Error #115: Keyword 'global' cannot be used in module files

Compiler says:

```
Keyword 'global' can not be used in module files
```

What it means in simple words:

You tried to declare a global variable inside a module.

How to fix it:

1. Move the `global` declaration to the main file.

Incorrect:

```
' myModule.bpm  
global x = 5
```

Correct:

```
' main.bp  
global x = 5  
Include "myModule.bpm"
```

◆ BLOCK 9: PROJECT ERRORS (#116–#121)

Error #116: Project name cannot contain more than 32 characters

Compiler says:

```
Project name can not contain more than 32 characters
```

What it means in simple words:

The project name is too long.

How to fix it:

1. Shorten the name to 32 characters or fewer.

Incorrect:

```
folder "prjs" "MyVeryVeryVeryLongProjectNameThatIsTooLong"
```

Correct:

```
folder "prjs" "MyShortProjectName"
```

Error #117: Project name cannot be void

Compiler says:

```
Project name can not be void
```

What it means in simple words:

You didn't specify a project name.

How to fix it:

1. Give the project a name.

Incorrect:

```
folder "prjs" ""
```

Correct:

```
folder "prjs" "MyProject"
```

Error #118: Project name must begin with A-Z, a-z

Compiler says:

```
Project name must begin with A-Z, a - z
```

What it means in simple words:

The project name starts with a digit or special character.

How to fix it:

1. Start the name with a letter.

Incorrect:

```
folder "prjs" "1Project"
```

Correct:

```
folder "prjs" "Project1"
```

Error #119: Project name can only contain letters, numbers, and _

Compiler says:

Project name can only **contain** letters **A-Z** and **a-z**, figures **0 - 9** and underscore character **_**

What it means in simple words:

The project name contains spaces or special characters.

How to fix it:

1. Use only letters, numbers, and .

Incorrect:

```
folder "prjs" "My Project!"  
folder "prjs" "My-Project"
```

Correct:

```
folder "prjs" "My_Project"
```

Error #120: The first parameter must be "prjs" or "sd"

Compiler says:

The first **parameter** must be "prjs" or "sd"

What it means in simple words:

You didn't specify **"prjs"** or **"sd"** as the first parameter after **folder**.

How to fix it:

1. Use **"prjs"** for robot memory or **"sd"** for SD card.

Incorrect:

```
folder "myFolder"
```

Correct:

```
folder "prjs" "myFolder"
```

Error #121: Keyword 'folder' must be before the main code

Compiler says:

Keyword 'folder' must be indicated **before the** beginning **of** main code

What it means in simple words:

You wrote **folder** after some code.

How to fix it:

1. Place **folder** at the very beginning of your program.

Incorrect:

```
LCD.Write(10, 10, "Hello")  
folder "prjs" "MyProject"
```

Correct:

```
folder "prjs" "MyProject"  
LCD.Write(10, 10, "Hello")
```

◆ BLOCK 10: .BPM MODULE AND EXTENDED CONSTRUCT ERRORS (#122–#171)

Error #122: Missing variable type after 'in/out'

Compiler says:

There must be a variable **type after** keyword **'in/out'** **in function definition**

What it means in simple words:

You wrote **in** or **out** but forgot the type.

How to fix it:

1. Always specify the type after **in** or **out**.

Incorrect:

```
Function MyFunction(in a)
    x = a
EndFunction
```

Correct:

```
Function MyFunction(in number a)
    x = a
EndFunction
```

Error #123: Variable type must be specified before the variable

Compiler says:

Variable **type must** be indicated before the variable **in function definition**

What it means in simple words:

You forgot to specify the variable type.

How to fix it:

1. Always put the type before the variable name.

Incorrect:

```
Function MyFunction(a)
    x = a
EndFunction
```

Correct:

```
Function MyFunction(in number a)
    x = a
EndFunction
```

Error #124: 'in/out' must follow a comma

Compiler says:

Keyword 'in/out' must be indicated after the comma in function definition

What it means in simple words:

You forgot `in` or `out` before the next parameter's type.

How to fix it:

1. Put `in` or `out` before the type of the next parameter.

Incorrect:

```
Function MyFunction(in number a, number b)
    x = a + b
EndFunction
```

Correct:

```
Function MyFunction(in number a, in number b)
    x = a + b
EndFunction
```

Error #125: 'in/out' must be before the variable type

Compiler says:

Keyword 'in/out' must be indicated before the variable type in function definition

What it means in simple words:

You forgot `in` or `out` before the type.

How to fix it:

1. Always put `in` or `out` before the type.

Incorrect:

```
Function MyFunction(number a)
    x = a
EndFunction
```

Correct:

```
Function MyFunction(in number a)
    x = a
EndFunction
```

Error #126: Variable name must follow the variable type

Compiler says:

Variable name must be indicated after the variable type in function definition

What it means in simple words:

You wrote `in number` but forgot the variable name.

How to fix it:

1. Always add the name after the type.

Incorrect:

```
Function MyFunction(in number)
    x = 0
EndFunction
```

Correct:

```
Function MyFunction(in number a)
    x = a
EndFunction
```

Error #127: 'in/out' and type must precede the variable

Compiler says:

```
Keyword 'in/out' and variable type must be indicated before the variable in
function definition
```

What it means in simple words:

You forgot `in/out` and the type before the variable.

How to fix it:

1. Use the format: `in/out type name`.

Incorrect:

```
Function MyFunction(a)
    x = a
EndFunction
```

Correct:

```
Function MyFunction(in number a)
    x = a
EndFunction
```

Error #128: Variable type not defined in function definition

Compiler says:

```
Variable type not defined in function definition
```

What it means in simple words:

You wrote **in** or **out** but forgot the type.

How to fix it:

1. Always specify the type after **in** or **out**.

Incorrect:

```
Function MyFunction(in a)  
    x = a  
EndFunction
```

Correct:

```
Function MyFunction(in number a)  
    x = a  
EndFunction
```

Error #129: Parameter has different type

Compiler says:

```
Parameter has different type
```

What it means in simple words:

You passed a parameter of the wrong type.

How to fix it:

1. Check the types of parameters you're passing.

Incorrect:

```
Function MyFunction(in number a)
    x = a * 2
EndFunction

MyFunction("5")
```

Correct:

```
Function MyFunction(in number a)
    x = a * 2
EndFunction

MyFunction(5)
```

Error #130: Output parameter in function call must be a variable

Compiler says:

In function call, output parameter can only be a variable

What it means in simple words:

You passed a value or expression as an **out** parameter instead of a variable.

How to fix it:

1. Use only variables for **out** parameters.

Incorrect:

```
Function MyFunction(out number result)
    result = 5
EndFunction

MyFunction(10)
```

Correct:

```
Function MyFunction(out number result)
    result = 5
EndFunction

MyFunction(x)
```

Error #131: Global variables cannot be used as parameters

Compiler says:

Parameters can **not** contain **references to global variables in function definition**

What it means in simple words:

You're trying to use a global variable as a parameter.

How to fix it:

1. Use local variable names for parameters.

Incorrect:

```
number x

Function MyFunction(in number x)
    x = 5
EndFunction
```

Correct:

```
number x

Function MyFunction(in number value)
    value = 5
EndFunction
```

Error #132: Boolean expression missing left side

Compiler says:

Boolean **expression missing** left side

What it means in simple words:

There's nothing on the left side of the logical operator.

How to fix it:

1. Add a variable or value on the left.

Incorrect:

```
If > 5 Then  
    LCD.Write(10, 10, "Hello")  
EndIf
```

Correct:

```
If x > 5 Then  
    LCD.Write(10, 10, "Hello")  
EndIf
```

Error #133: Boolean expression missing right side

Compiler says:

Boolean **expression missing** right side

What it means in simple words:

There's nothing on the right side of the logical operator.

How to fix it:

1. Add a variable or value on the right.

Incorrect:

```
If x > Then
    LCD.Write(10, 10, "Hello")
EndIf
```

Correct:

```
If x > 5 Then
    LCD.Write(10, 10, "Hello")
EndIf
```

Error #134: Errors in For initialization line

Compiler says:

```
Errors in For initialization line
```

What it means in simple words:

There is a syntax error in the **For** line.

How to fix it:

1. Use the correct format: **For variable = start To end** .

Incorrect:

```
For i 1 To 10
```

Correct:

```
For i = 1 To 10
    LCD.Write(10 * i, 10 * i, i)
EndFor
```

Error #135: 'For' must be followed by a variable and assignment

Compiler says:

'For' must be followed by a variable and the value assigned to it

What it means in simple words:

You didn't specify the variable or assign a starting value.

How to fix it:

1. Use the format: `For variable = start To end`.

Incorrect:

```
For = 1 To 10  
For i To 10
```

Correct:

```
For i = 1 To 10  
    LCD.Write(10 * i, 10 * i, i)  
EndFor
```

Error #136: In For loop, variable must contain a number

Compiler says:

In For cycle, variable must contain a number

What it means in simple words:

The loop variable doesn't contain a number.

How to fix it:

1. Use numbers for the start and end values.

Incorrect:

```
For i = "1" To 10  
    LCD.Write(10 * i, 10 * i, i)  
EndFor
```

Correct:

```
For i = 1 To 10
    LCD.Write(10 * i, 10 * i, i)
EndFor
```

Error #137: 'Then' must be at the end of the line

Compiler says:

'Then' must be indicated at the end of the line

What it means in simple words:

You forgot **Then** at the end of the **If** line.

How to fix it:

1. Always put **Then** at the end of the condition line.

Incorrect:

```
If x > 5
    LCD.Write(10, 10, "Hello")
EndIf
```

Correct:

```
If x > 5 Then
    LCD.Write(10, 10, "Hello")
EndIf
```

Error #138: Missing logical condition

Compiler says:

Missing logical condition

What it means in simple words:

You wrote `If Then` but didn't specify the condition.

How to fix it:

1. Write the condition between `If` and `Then`.

Incorrect:

```
If Then
    LCD.Write(10, 10, "Hello")
EndIf
```

Correct:

```
If x > 5 Then
    LCD.Write(10, 10, "Hello")
EndIf
```

Error #139: Missing imported module file name

Compiler says:

```
Missing imported module file name
```

What it means in simple words:

You wrote `Import` but didn't specify a file name.

How to fix it:

1. Add the file name.

Incorrect:

```
Import
```

Correct:

```
Import "myModule"
```

Error #140: Imported modules cannot contain 'include'

Compiler says:

Imported **module** files can **not** contain **'include'** inclusions

What it means in simple words:

You used **Include** inside an imported module.

How to fix it:

1. Remove **Include** from imported modules.

Incorrect:

```
' myModule.bpm
Include "helpers"
```

Correct:

```
' main.bp
Include "helpers"
Import "myModule"
```

Error #141: Imported modules cannot contain keyword 'folder'

Compiler says:

Imported **module** files can **not** contain keyword **'folder'**

What it means in simple words:

You used **folder** inside an imported module.

How to fix it:

1. Remove **folder** from imported modules.

Incorrect:


```
' myModule.bpm  
folder "prjs" "MyProject"
```

Correct:

```
' main.bp  
folder "prjs" "MyProject"  
Import "myModule"
```

Error #142: Procedures are not allowed in .bpm files

Compiler says:

Invalid **procedure** definition in .bpm module files

What it means in simple words:

You tried to declare a procedure in a **.bpm** file.

How to fix it:

1. Use **Function** instead of **Sub** in **.bpm** files.

Incorrect:

```
' myModule.bpm  
Sub MyProcedure  
    LCD.Write(10, 10, "Hello")  
EndSub
```

Correct:

```
' myModule.bpm  
Function MyFunction()  
    LCD.Write(10, 10, "Hello")  
EndFunction
```

Error #143: .bpm files can only contain functions and properties

Compiler says:

.bpm **module** files can **only** contain **function** definitions and properties

What it means in simple words:

There is something in the **.bpm** file that isn't a function or property.

How to fix it:

1. Remove everything except **Function** and properties.

Incorrect:

```
' myModule.bpm  
x = 5
```

Correct:

```
' myModule.bpm  
Function MyFunction()  
    x = 5  
EndFunction
```

Error #144: Global variables cannot be used in .bpm

Compiler says:

References to **global** variables can **not** be used **in** .bpm modules

What it means in simple words:

You're trying to use a global variable inside a **.bpm** file.

How to fix it:

1. Pass values as parameters instead.

Incorrect:

```
' main.bp
global x

' myModule.bpm
Function MyFunction()
    y = x + 10
EndFunction
```

Correct:

```
' main.bp
global x

' myModule.bpm
Function MyFunction(in number value)
    y = value + 10
EndFunction
```

Error #145: Global labels cannot be used in .bpm

Compiler says:

References to global goto labels can not be used in .bpm modules

What it means in simple words:

You're trying to use a `Goto` with a label from another file.

How to fix it:

1. Don't use `Goto` in `.bpm` files.

Incorrect:

```
' main.bp
myLabel:

' myModule.bpm
```

```
Function MyFunction()  
    Goto myLabel  
EndFunction
```

Correct:

```
' myModule.bpm  
Function MyFunction()  
    If condition Then  
        ' code  
    EndIf  
EndFunction
```

Error #146: Module property must consist of type and name

Compiler says:

A module **property** declaration must only consist of the **property type and name**

What it means in simple words:

You declared a module property incorrectly.

How to fix it:

1. Use the format: **type name** .

Incorrect:

```
Property myProperty  
number myProperty = 5
```

Correct:

```
number myProperty
```

Error #147: Module property: type and name

Compiler says:

A module **property** must consist of its **type and** name

What it means in simple words:

You forgot the type or name of the property.

How to fix it:

1. Always specify both type and name.

Incorrect:

```
Property  
myProperty
```

Correct:

```
number myProperty
```

Error #148: Property type can only be number, number[], string, string[]

Compiler says:

A **property** can only be of **number, number[], string, or string[]** type

What it means in simple words:

You used an invalid type for a property.

How to fix it:

1. Use only allowed types.

Incorrect:

```
boolean flag
```

Correct:

```
number flag
```

Error #149: Property with this name is already defined

Compiler says:

```
A property with this name has already been defined in the module
```

What it means in simple words:

You're trying to declare two properties with the same name.

How to fix it:

1. Use a different name.

Incorrect:

```
number x  
number x
```

Correct:

```
number x  
number y
```

Error #150: Property cannot be declared inside a module method

Compiler says:

```
A property cannot be declared inside a module method (function)
```

What it means in simple words:

You're trying to declare a property inside a function.

How to fix it:

1. Declare properties outside functions.

Incorrect:

```
Function MyFunction()  
    number x  
EndFunction
```

Correct:

```
number x  
  
Function MyFunction()  
    y = x  
EndFunction
```

Error #151: Only 'private' keyword is allowed on this line

Compiler says:

The 'private' keyword is the only admissible keyword in the line

What it means in simple words:

You wrote extra text on the same line as `private`.

How to fix it:

1. The line with `private` must contain only that word.

Incorrect:

```
private number x
```

Correct:

```
private  
number x
```

Error #152: Private property can only be called in the owning module

Compiler says:

Calling a **private property** is allowed only **in** the module that owns **this property**

What it means in simple words:

You're trying to access a private property from another module.

How to fix it:

1. Use private properties only inside their own module.

Incorrect:

```
' module1.bpm
private
number x

' module2.bpm
value = Module1.x
```

Correct:

```
' module1.bpm
Function GetX(out number a)
    a = x
EndFunction

private
number x

' module2.bpm
value = Module1.GetX(a)
```

Error #153: Private method can only be called in the owning module

Compiler says:

Calling a **private method** is allowed only in the module owner of this method

What it means in simple words:

You're trying to call a private function from another module.

How to fix it:

1. Use private functions only inside their own module.

Incorrect:

```
' module1.bpm
private
Function Internal()
    x = 5
EndFunction

' module2.bpm
value = Module1.Internal()
```

Correct:

```
' module1.bpm
Function Public()
    Internal()
EndFunction

private
Function Internal()
    x = 5
EndFunction

' module2.bpm
value = Module1.Public()
```

Error #154: Variable name conflicts with property name in module

Compiler says:

The **variable name** in the **module function** description **is** the same **as** the **module** property **name**

What it means in simple words:

A function parameter has the same name as a module property.

How to fix it:

1. Use a different name for the parameter.

Incorrect:

```
number x

Function MyFunction(x)
    y = x + 10
EndFunction
```

Correct:

```
number x

Function MyFunction(value)
    y = value + 10
EndFunction
```

ERRORS: BREAK, CONTINUE, RETURN AND OTHERS (#155–#171)

Error #155: Only one 'Break' per line

Compiler says:

There can **only** be **one** keyword "Break" **per** line

What it means in simple words:

You wrote **Break** multiple times in one line.

How to fix it:

1. Keep only one **Break** per line.

Incorrect:

```
Break Break
```

Correct:

```
Break
```

Error #156: 'Break' can only be used inside For and While loops

Compiler says:

Keyword "Break" can be used **in** the middle of For...EndFor **and** While...EndWhile

What it means in simple words:

You're trying to use **Break** outside a loop.

How to fix it:

1. Use **Break** only inside loops.

Incorrect:

```
If x > 5 Then  
    Break  
EndIf
```

Correct:

```
For i = 1 To 10
  If i > 5 Then
    Break
  EndIf
EndFor
```

Error #157: Only one 'Continue' per line

Compiler says:

There can **only** be **one** keyword "Continue" **per** line

What it means in simple words:

You wrote **Continue** multiple times in one line.

How to fix it:

1. Keep only one **Continue** per line.

Incorrect:

```
Continue Continue
```

Correct:

```
Continue
```

Error #158: 'Continue' can only be used inside For and While loops

Compiler says:

Keyword "Continue" can be used **in** the middle of For...EndFor **and** While...EndWhile

What it means in simple words:

You're trying to use **Continue** outside a loop.

How to fix it:

1. Use `Continue` only inside loops.

Incorrect:

```
If x > 5 Then  
    Continue  
EndIf
```

Correct:

```
For i = 1 To 10  
    If i = 5 Then  
        Continue  
    EndIf  
EndFor
```

Error #159: Only one 'Return' per line

Compiler says:

There can **only** be **one** keyword "Return" **per** line

What it means in simple words:

You wrote `Return` multiple times in one line.

How to fix it:

1. Keep only one `Return` per line.

Incorrect:

```
Return Return
```

Correct:

```
Return 5
```

Error #160: 'Return' can only be used inside Sub and Function

Compiler says:

Keyword "Return" can be used in the middle of Sub...EndSub and Function...EndFunction

What it means in simple words:

You're trying to use **Return** outside a procedure or function.

How to fix it:

1. Use **Return** only inside **Sub** or **Function**.

Incorrect:

```
If x > 5 Then
    Return
EndIf
```

Correct:

```
Function MyFunction()
    If x > 5 Then
        Return 5
    EndIf
EndFunction
```

Error #161: Property with this name is not defined in the module

Compiler says:

A property with this name is not defined in the module

What it means in simple words:

You're trying to access a property that doesn't exist in the module.

How to fix it:

1. Check the spelling.
2. Make sure the property is declared in the module.
3. If it's private, it's only accessible inside the module.

Incorrect:

```
' myModule.bpm
number x

' module2.bpm
value = Module1.y    ' property y does not exist
```

Correct:

```
' myModule.bpm
number x
number y

' module2.bpm
value = Module1.y    ' property y is declared
```

END OF REFERENCE

Total errors: 161

Date: 08.2026

This reference is based on compiler messages from Clever (Small Basic Plus).
For questions or suggestions, please contact the author.

...